49m left

ALL

19

19. Find Longest Ride

Given the dataset of taxi rides completed in a pandas dataframe, process the dataframe as follows:

- Drop all the rows where the pickup time or dropoff time is missing.
- Find the longest ride (on basis of duration) for each pickup month (YYYY-MM).
- Sort the resulting dataframe by the pickup month.

The given dataframe consists of four columns:

- id - unique trip id
- vendor_id - vendor id
- pickup_datetime - when the ride started
- dropoff_datetime - when the ride ended.

Consider, for example, you are given the following dataframe:

id	vendor_id	pickup_datetime	dropoff_datetime
id01234	3	2016-06-06 06:06:20	2016-06-06 08:00:00
id01434	4	2016-06-09 06:06:20	2016-06-09 06:07:20
id13234	3		2016-07-06 03:06:10

After performing the steps:

pickup_month	id

Language: Python 3 | Environment | Autocomplete Ready

```
1  #!/bin/python3 ...
6
7  #
8  # Complete the 'longestRide' function below.
9  #
10 # The function is expected to return a dataframe.
11 #
12
13 def longestRide(df):
14     # Write your code here
15
16 if __name__ == '__main__': ...
```

Line: 6 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

41m left

ALL

id01234	3	2016-06-06 06:06:20	2016-06-06 08:00:00
id01434	4	2016-06-09 06:06:20	2016-06-09 06:07:20
id13234	3		2016-07-06 03:06:10

After performing the steps:

pickup_month	id
2016-06	id01234

Function Description
Complete the function *longestRide* in the editor below.

longestRide has the following parameter(s):
df: a pandas dataframe

Constraints

pandas dataframe: the results of processing *df*

Constraints

- 1 ≤ the number of rows in the dataframe ≤ 1000.
- It is guaranteed that the resulting dataframe consists of at least one row with no nulls.

▸ **Input Format For Custom Testing**

▾ **Sample Case 0**

Language: Python 3 | Environment | Autocomplete Ready

```
12
13  def longestRide(df):
14      # Write your code here
15      Id = df['id']
16      vendor = df['vendor_id']
17      pickup = df['pickup_datetime']
18      dropoff_datetime = df['dropoff_datetime']
19
20      print(Id[0][1])
21      print(vendor)
22      return df
23
```

Line: 20 Col: 20

Test Results | **Custom Input** | **Run Code** | **Run Tests** | **Submit**

Compiled successfully. **Run all test cases**

Debug output

```
0    id0219696
1    id1372182
2    id3569980
3    id2858528
Name: id, dtype: object
0    72
```

41m left

ALL

19

Constraints

pandas dataframe: the results of processing *df*

Constraints

- 1 ≤ the number of rows in the dataframe ≤ 1000.
- It is guaranteed that the resulting dataframe consists of at least one row with no nulls.

▶ **Input Format For Custom Testing**

▼ **Sample Case 0**

Sample Input For Custom Testing

```
id,vendor_id,pickup_datetime,dropoff_datetime
id0219696,72,2016-06-06 06:06:20,2016-06-06 06:13:34
id1372182,80,2016-02-07 19:18:49,
id3569980,69,2016-06-14 00:26:11,2016-06-14 00:34:09
id2858528,29,,
```

Sample Output

```
pickup_month,id
2016-06,id3569980
```

Explanation

After dropping null rows, only 2 trips remain in 2016-06. The duration is calculated and the pickup_month and id for the longest ride is returned.

▶ **Sample Case 1**

Language: Python 3 | Environment | Autocomplete Ready

```
12
13  def longestRide(df):
14      # Write your code here
15      Id = df['id']
16      vendor = df['vendor_id']
17      pickup = df['pickup_datetime']
18      dropoff_datetime = df['dropoff_datetime']
19
20      print(Id[0][1])
21      print(vendor)
22      return df
23
```

Line: 20 Col: 20

Test Results | **Custom Input** | **Run Code** | **Run Tests** | **Submit**

Compiled successfully. **Run all test cases**

Debug output

```
0    id0219696
1    id1372182
2    id3569980
3    id2858528
Name: id, dtype: object
0    72
```

41m left

ALL

19

Sample Input For Custom Testing

```
id,vendor_id,pickup_datetime,dropoff_datetime
id0219696,72,2016-06-06 06:06:20,2016-06-06 06:13:34
id1372182,80,2016-02-07 19:18:49,
id3569980,69,2016-06-14 00:26:11,2016-06-14 00:34:09
id2858528,29,,
```

Sample Output

```
pickup_month,id
2016-06,id3569980
```

Explanation

After dropping null rows, only 2 trips remain in 2016-06. The duration is calculated and the pickup_month and id for the longest ride is returned.

▼ Sample Case 1

Sample Input For Custom Testing

```
id,vendor_id,pickup_datetime,dropoff_datetime
id2714955,83,,2016-02-29 08:12:29
id0418897,42,2016-06-01 07:46:35,
id2366946,61,2016-02-14 14:15:42,2016-02-14 14:21:32
id1943471,67,,2016-04-24 11:47:33
id2366984,62,2016-03-14 14:15:42,2016-03-14 14:21:32
```

Sample Output

```
pickup_month,id
2016-02,id2366946
2016-03,id2366984
```

Explanation

After dropping null rows, only 2 trips remain - one each for 2016-02 & 2016-03. The pickup_month and id are returned for each month, sorted in increasing order of date.

Language: Python 3 | Environment | Autocomplete Ready

```
12
13  def longestRide(df):
14      # Write your code here
15      Id = df['id']
16      vendor = df['vendor_id']
17      pickup = df['pickup_datetime']
18      dropoff_datetime = df['dropoff_datetime']
19
20      print(Id[0][1])
21      print(vendor)
22      return df
23
```

Line: 20 Col: 20

Test Results | Custom Input | Run Code | Run Tests | Submit

Compiled successfully. Run all test cases

Debug output

```
0    id0219696
1    id1372182
2    id3569980
3    id2858528
Name: id, dtype: object
0    72
```

58m left

ALL

13

14

15

16

17

18

19

20

20. C++: The Event

A major event is taking place at a mall with multiple entry gates. Attendees arrive with friends and specify the gate they will use.

You have two integer arrays of length *n*:

- *friends*: Represents the number of friends each attendee is bringing.
- *gates*: Represents the gate each attendee will enter.

Determine the total number of people entering through each gate.

Example

friends = [1, 2, 3]
gates = [1, 1, 2]
m = 2

For three attendees with two gates:

1. The first attendee, with 1 friend, enters through Gate 1.
2. The second attendee, with 2 friends, also chooses Gate 1.
3. The third attendee, with 3 friends, enters through Gate 2.

In this case:

- Gate 1 sees 5 people (first attendee with 1 friend and second with 2 friends)

Language C++11 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h> ···
9
10 /*
11  * Complete the 'getGatesCount' function below.
12  *
13  * The function is expected to return an INTEGER_ARRAY.
14  * The function accepts following parameters:
15  *  1. INTEGER_ARRAY friends
16  *  2. INTEGER_ARRAY gates
17  *  3. INTEGER m
18  */
19
20 vector<int> getGatesCount(vector<int> friends, vector<int> gates, int m) {
21
22 }
23
24 int main() ···
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit

58m left

ALL

13

14

15

16

17

18

19

20

- Gate 2 sees 4 people (third attendee with 3 friends).

Return `[5, 4]`

Function Description
Complete the function *getGatesCount* in the editor below.

Function Parameters
int friends[n]: friends[i] denotes the number of friends accompanying the i^{th} attendee
int gates[n]: gates[i] indicates the gate through which the i^{th} attendee and their friends enter
int m: the number of gates

Returns
int[m]: An array representing the total number entering from each gate

Constraints

- $1 \le n, m \le 100000$
- $_1 \le friends[i] \le 1000$
- $1 \le gate[i] \le m$

▸ **Input Format for Custom Testing**

▾ **Sample Case 0**

Sample Input 0

STDIN Function

Language C++11 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h> ···
9
10 /*
11  * Complete the 'getGatesCount' function below.
12  *
13  * The function is expected to return an INTEGER_ARRAY.
14  * The function accepts following parameters:
15  *  1. INTEGER_ARRAY friends
16  *  2. INTEGER_ARRAY gates
17  *  3. INTEGER m
18  */
19
20 vector<int> getGatesCount(vector<int> friends, vector<int> gates, int m) {
21
22 }
23
24 int main() ···
```

Line: 9 Col: 1

Test Results | Custom Input

Run Code | Run Tests | Submit

58m left

ALL

13

14

15

16

17

18

19

20

▶ **Input Format for Custom Testing**

▼ **Sample Case 0**

Sample Input 0

```
STDIN   Function
-----   --------
3       friends[] size n = 3
1       friends = [1, 2, 3]
2
3
3       gates[] size n = 3
1       gates = [1, 2, 3]
2
3
3       m = 3
```

Sample Output 0

```
2
3
4
```

Explanation

In this test case, there are three attendees and three gates.

- Attendee 1 arrives with 1 friend, and they enter through Gate 1.
- Attendee 2 arrives with 2 friends, and they enter through Gate 2.
- Attendee 3 arrives with 3 friends, and they enter through Gate 3.

The breakdown for each gate is:

Language C++11 | Environment | Autocomplete Ready

```
1  > #include <bits/stdc++.h> ···
9
10 /*
11  * Complete the 'getGatesCount' function below.
12  *
13  * The function is expected to return an INTEGER_ARRAY.
14  * The function accepts following parameters:
15  *  1. INTEGER_ARRAY friends
16  *  2. INTEGER_ARRAY gates
17  *  3. INTEGER m
18  */
19
20 vector<int> getGatesCount(vector<int> friends, vector<int> gates, int m) {
21
22 }
23
24 > int main() ···
```

Line: 9 Col: 1

Test Results **Custom Input** **Run Code** **Run Tests** **Submit**

58m left

The breakdown for each gate is:

- Gate 1: Attendee 1 and one friend enter.
- Gate 2: Attendee 2 and two friends enter.
- Gate 3: Attendee 3 and three friends enter.

▼ Sample Case 1

Sample Input 1

```
3
3
5
1
3
1
1
3
3
```

Sample Output 1

```
10
0
2
```

Explanation

Three attendees and three gates.

- Attendee 1 arrives with 3 friends, and they enter Gate 1.
- Attendee 2 arrives with 5 friends, and they enter Gate 1.
- Attendee 3 arrives with 1 friend and they enter Gate 3.

ALL

13 14 15 16 17 18 19 20

Language C++11 | Environment | Autocomplete Ready

```
1  #include <bits/stdc++.h> ···
9
10 /*
11  * Complete the 'getGatesCount' function below.
12  *
13  * The function is expected to return an INTEGER_ARRAY.
14  * The function accepts following parameters:
15  *  1. INTEGER_ARRAY friends
16  *  2. INTEGER_ARRAY gates
17  *  3. INTEGER m
18  */
19
20 vector<int> getGatesCount(vector<int> friends, vector<int> gates, int m) {
21
22 }
23
24 int main() ···
```

Line: 9 Col: 1

Test Results | Custom Input | Run Code | Run Tests | Submit